OPSNEXUS

Unified Operations Intelligence for Data Pipelines, Data Trust, Requests, and Platform Cost

Project Documentation

Track 04 — Work Behind the Work

Presented by Team Nexus

A single operational view that helps engineering and analytics teams observe activity, investigate issues, validate evidence, and make better-informed decisions.

1. Executive Summary

OpsNexus is a unified operations intelligence layer designed to bring pipeline monitoring, data-quality signals, data-request support, and cloud-cost context into one dashboard.

Operational information is often spread across separate platforms. Teams must switch between pipeline run histories, quality checks, database tables, request channels, and billing views to understand what happened and what to do next.

OpsNexus brings these signals together through a React dashboard, a Flask REST API, Exasol Personal as the analytical data layer, Azure integrations, and an optional Ollama/Llama 3 workflow for natural-language assistance.

> **Core idea: turn scattered operational signals into structured, evidence-backed findings that are easier to understand and act on.**

What the project aims to deliver

- A single interface for operational visibility.
- Consistent access to pipeline, quality, request, and cost information.
- SQL-driven analytical views and deterministic rule evaluation in Exasol.
- Optional AI assistance for schema-aware questions and data requests.
- A reproducible local demonstration using Exasol Personal and documented setup steps.

2. Team and Project Context

Team: Team Nexus

Event: Exathon 2026

Track: Track 04 — Work Behind the Work

S. No.	Name	Role	LinkedIn URL
1	Vyshhnavi Ndd	Team Leader	Click here
2	Logesh S	Team Member	Click here

Team mission

The team's goal is to make the operational work behind data products more visible. Rather than treating pipeline health, data trust, user requests, and platform spend as unrelated concerns, OpsNexus presents them as connected parts of day-to-day data operations.

Working principles

- Visibility: surface operational status and important signals in one place.
- Trust: show quality indicators and evidence rather than relying on unexplained numbers.
- Actionability: help users investigate the signal and identify a reasonable next step.
- Reproducibility: keep the implementation and setup instructions accessible in the project repository.

3. Problem Statement

Operational work is invisible until something breaks.

Data engineers, analytics engineers, platform teams, and business stakeholders lose time connecting signals scattered across pipelines, quality checks, data requests, and cloud spending. The resulting fragmentation can delay diagnosis, reduce confidence in data, and create avoidable manual work.

Four connected pain points

1 — Pipelines fail silently

Failures and delays can go unnoticed until downstream workflows are affected. Teams need a clear view of run status, activity details, metadata, and trigger information.

2 — Data trust is difficult to verify

Users may see metrics without being able to quickly verify freshness, rejected rows, completeness, or overall quality status.

3 — Data requests stall

Finding the correct table, understanding its schema, and routing a request can depend on manual help from a specialist.

4 — Cost outruns usage context

Spending is difficult to interpret when billing information is separated from operational usage and pipeline activity.

4. Who Has the Problem, and Why It Matters

Primary users

Data engineers

Need to monitor pipeline runs, investigate failures, understand dependencies, and confirm whether data movement completed as expected.

Analytics engineers and data analysts

Need to assess whether datasets are complete and reliable before using them in analysis or reports.

Platform and cloud operations teams

Need to connect service usage and operational activity with cost trends, resource consumption, and budget considerations.

Business and data-request stakeholders

Need to discover available datasets, understand data at a high level, and make requests without requiring every question to become a manual engineering task.

Impact of the problem

- Longer time to detect and diagnose operational issues.
- Reduced confidence in downstream reports when quality context is missing.
- Repeated interruptions to engineers for schema and dataset questions.
- Limited ability to explain cost changes using operational context.

OpsNexus is designed to reduce these gaps by consolidating visibility and presenting evidence that supports investigation. The intended benefit is faster understanding and better decisions; actual time or cost savings should be measured during a longer pilot rather than assumed.

5. Solution Overview

OpsNexus provides one operational console with four connected modules. Each module addresses a distinct problem while contributing to the same observe–investigate–decide–improve workflow.

Pipeline Watch — Detect

Monitor pipeline runs, identify failed or delayed executions, and inspect available execution details and metadata.

Data Trust — Verify

Review quality results, freshness indicators, and valid-versus-invalid record information to understand whether data is ready for use.

Request Hub — Unblock

Help users discover datasets and organize structured data requests. The optional AI workflow can assist with schema-aware questions.

Cost Lens — Optimize

Explore cost trends and relate spending to operational context. Cost information should be interpreted with its time range, currency, and source status visible.

Operating loop: Observe → Investigate → Decide → Improve.

Design principle

The dashboard is the common entry point; the service layer retrieves and shapes information; Exasol supports analytical querying; external platforms remain authoritative for their own records.

6. System Architecture — End-to-End Flow

The architecture diagram below shows the intended relationship between the user-facing dashboard, API services, analytical storage, cloud integrations, and optional AI assistance.

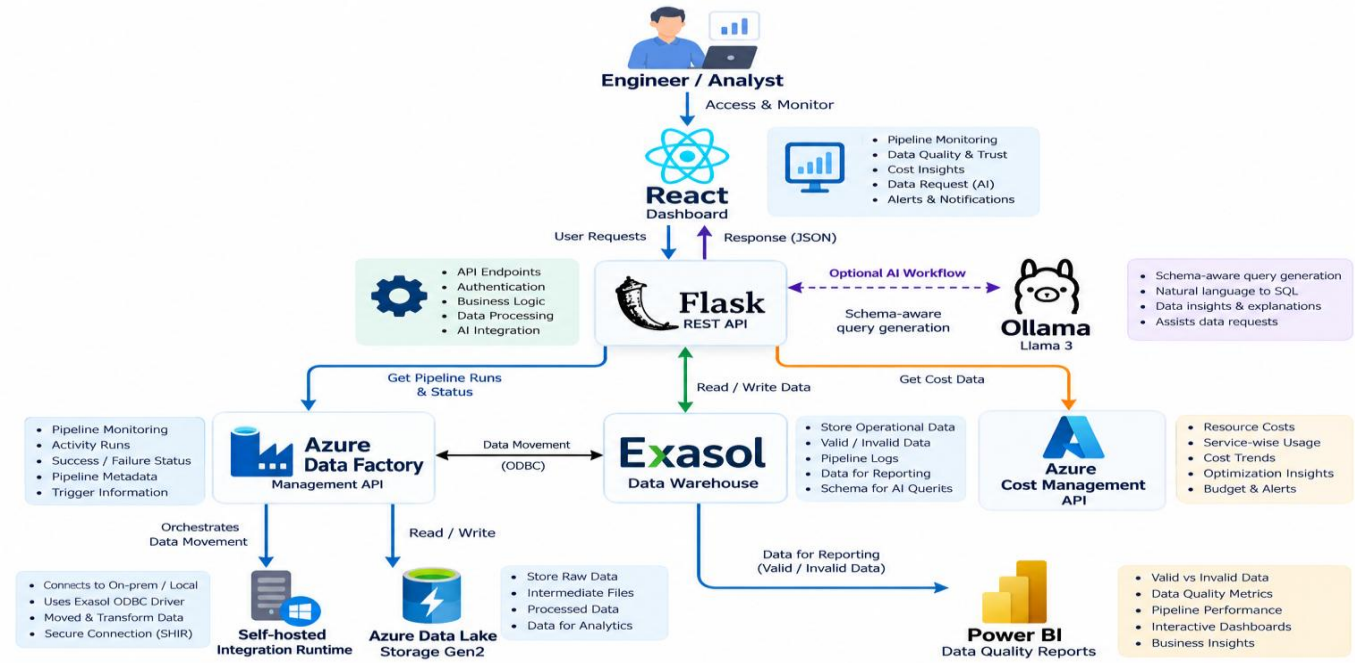


Figure 1. OpsNexus architecture and data-flow diagram supplied with the project materials.

The React dashboard sends user requests to the Flask REST API and renders JSON responses. Flask connects to Exasol for analytical data and to Azure APIs for pipeline and cost information. Azure Data Factory orchestrates data movement, with a self-hosted Integration Runtime supporting connectivity to local or on-premises systems where configured. Azure Data Lake Storage Gen2 can hold raw, intermediate, and processed files. Power BI is shown as a reporting destination for data-quality and operational reporting.

The diagram is a conceptual architecture view. Specific connectors, permissions, refresh behavior, and deployed integrations should be verified against the actual environment before production use.

7. Technical Design and Data Flow

Presentation layer — React

The React application provides the operator-facing dashboard and its modules: overview, pipelines, trust, cost, chat, requests, and database exploration. The frontend calls backend endpoints and presents returned data as metrics, tables, and reports.

Application layer — Flask REST API

Flask acts as the service boundary between the frontend, Exasol, Azure APIs, and optional AI services. It centralizes request handling, data shaping, integration logic, and error responses.

Analytical layer — Exasol Personal

Exasol is used as the analytical core for operational datasets, SQL views, aggregations, and rule-based findings. In the supplied pitch deck, the schema and view examples are named OPS_INTEL and OPS_INTEL.V_FINDINGS.

External integrations — Azure

Azure Data Factory provides pipeline-run and activity context through its management interface. Azure Cost Management provides cost data. Azure Data Lake Storage Gen2 is shown as a storage component in the data movement path.

Typical request flow

1. An engineer opens a dashboard module or asks a question.
2. React sends a request to a Flask endpoint.
3. Flask retrieves data from Exasol and/or the relevant external API.
4. The service returns a structured response, including a clear error or degraded state when data cannot be retrieved.
5. React displays the result so the user can investigate or take the next step.

Reliability principle: when a source API is unavailable, show a clear degraded state and the last successful refresh where available.

8. Effective Use of Exasol Personal

Exasol Personal serves as the local analytical database for the project demonstration. It provides a place to consolidate operational records and execute SQL-based analysis without making the dashboard itself responsible for every analytical calculation.

How Exasol contributes

- Stores and serves the unified operational dataset used by the intelligence layer.
- Supports SQL views that shape source data into analysis-ready structures.
- Runs deterministic rules and aggregations to identify operational findings.
- Provides structured evidence that can be shown alongside an identified issue.
- Supports repeatable local validation against known scenarios.

Evidence reported in the supplied pitch deck

- **17 live findings returned by OPS_INTEL.V_FINDINGS.**
- **7 deterministic operational rule families.**
- **8/8 scenario targets matched.**
- **Evidence audit passed for all live findings in the documented local check.**

Schema and view terminology

The source deck describes OPS_INTEL as the Exasol schema and V_FINDINGS as a view. A schema groups related database objects; a view presents a query-defined representation of data.

These figures are the validation results stated in the uploaded pitch material, not a guarantee of performance in every deployment. Keep the evidence record and test date with any future published benchmark.

9. Technical Excellence — Technology Stack

React — dashboard and user experience

Provides the interactive web interface, operational modules, and visual presentation of backend responses.

Flask — REST API and integration layer

Exposes endpoints, handles application logic, and coordinates access to analytical data and external services.

Exasol Personal — analytical database

Executes SQL views, aggregations, and deterministic operational rules over structured project data.

Azure — operational data sources and movement

Azure Data Factory supplies pipeline monitoring context; Azure Cost Management supplies cost data; Azure Data Lake Storage Gen2 is included in the depicted data movement architecture.

Ollama and Llama 3 — optional local AI workflow

The diagram presents Ollama running Llama 3 as an optional assistant for schema-aware query generation, natural-language-to-SQL support, explanations, and data-request assistance.

Why this separation matters

- The frontend focuses on presenting information.
- The API layer isolates integration details and response formatting.
- Exasol handles analytical SQL workloads.
- Cloud services remain the source of their own operational and billing records.
- AI assistance is optional and should not replace validation of generated SQL or evidence.

10. AI Workflow and Guardrails

The supplied architecture shows an optional AI workflow between Flask and Ollama (Llama 3). The intended role is to help users ask questions about available data in natural language and receive schema-aware assistance.

Potential workflow

6. A user asks a question through the dashboard.
7. The service retrieves relevant schema metadata that the user is permitted to access.
8. The model proposes a query or explanation grounded in that schema context.
9. The application validates the generated query before execution.
10. The response is returned with a clear explanation and, where possible, supporting data or evidence.

Recommended guardrails

- Use read-only database credentials for exploratory natural-language queries wherever possible.
- Allowlist schemas, views, and query operations; reject unsafe or unsupported SQL.
- Apply query limits and timeouts to prevent expensive or unbounded execution.
- Do not treat model-generated statements as trusted until validated.
- Return uncertainty and source limitations honestly; do not invent columns or results.
- Avoid sending secrets or unnecessary sensitive records to the model.

AI assists with discovery and explanation; database permissions, validation, and deterministic rules remain the control points.

11. Solution Design and User Experience

OpsNexus is organized around the questions an operator needs to answer: What ran? Can the data be trusted? What information is available? What is the platform costing?

Navigation and information architecture

- Overview: summarize operational status and key indicators.
- Pipelines: inspect runs, statuses, activities, and metadata.
- Trust: review quality results and valid/invalid data indicators.
- Cost: explore cost trends and service-level usage context.
- Chat and Requests: support natural-language discovery and structured requests.
- Database: explore the analytical data available to the application.

Experience principles

- Use consistent names, units, date ranges, and status labels.
- Distinguish missing data from zero values and successful results.
- Show loading, empty, error, and degraded states explicitly.
- Make tables readable with stable column widths, clear alignment, and sensible overflow behavior.
- Expose enough evidence and source context to help users verify important findings.

A good operational interface does not only show a metric; it helps the user understand what the metric represents, how current it is, and what to inspect next.

12. Validation, Testing, and Evidence

The uploaded project pitch describes validation of analytical rules, database outputs, API behavior, and evidence-backed findings.

Reported local validation results

- **17 live findings returned by the findings view.**
- **7 rule families evaluated.**
- **8/8 scenario targets matched, including positive scenarios, a healthy baseline, and an inconclusive case.**
- **Evidence audit passed for all live findings in the documented check.**

API verification areas listed in the source

- Health check.
- Pagination.
- Seven finding-type filters.
- Scenario filtering.
- Empty results.
- Detail/list consistency.

Suggested repeatable test checklist

- Start Exasol Personal and verify the database connection.
- Start the Flask service and verify its health endpoint.
- Build and launch the React frontend.
- Check success, empty, timeout, and authorization-error responses.
- Compare findings shown in the UI with the underlying Exasol view.
- Record the run date, configuration, test cases, and observed outputs.

The source pitch cites a local verification record dated 19 October 2026. Re-run the checks and update this date before claiming that the current repository state has passed them.

13. Documentation and Reproducibility

A reproducible project should let another developer understand the architecture, configure dependencies, launch the application, and verify the expected behavior without relying on undocumented steps.

Local run sequence

11. From the repository root, start the existing Exasol Personal deployment if it is not already running.
12. Build the frontend after making frontend changes by running npm install and npm run build in the frontend directory.
13. Start the integrated Flask application from the repository root using the project's configured Python environment.
14. Open the dashboard and verify the health endpoint and core modules.
15. Confirm that the displayed data matches the configured Exasol database and any connected Azure services.

What the repository documentation should include

- Prerequisites and supported local environment.
- Environment-variable names and safe example values, with secrets excluded.
- Database setup, schema/view creation, and seed or scenario data instructions.
- Frontend and backend startup commands.
- API endpoint list with sample request/response shapes.
- Troubleshooting for database connectivity, Azure permissions, and unavailable services.
- Validation commands and expected results.

Never commit client secrets, tokens, connection passwords, private keys, or local deployment binaries to the public repository.

14. Open-Source Quality and Project Links

OpsNexus is associated with a public GitHub repository. The repository should be the canonical place for source code, setup instructions, issue tracking, and contribution guidance.

Repository

github.com/vyshhnavi-ndd/OpsNexus — source code and project files

Open-source readiness checklist

- Include a clear README with project purpose, architecture, screenshots or diagrams, prerequisites, and setup instructions.
- Add a license file and ensure that dependencies and included assets are compatible with that license.
- Keep secrets, generated credentials, local databases, and large deployment binaries out of version control.
- Use a .env.example file containing variable names and safe placeholders.
- Document how to report issues and propose changes.
- Provide a small validation checklist so contributors can confirm their changes.
- Keep the architecture diagram and API documentation synchronized with the code.

Professional profile links

Individual LinkedIn URLs and an official team profile URL were not included in the supplied materials. Add verified profile links here before distributing the final document; no profile links have been fabricated .Suggested README sections

Overview • Problem statement • Architecture • Tech stack • Prerequisites • Installation • Configuration • Running locally • API reference • Validation • Known limitations • License • Contributors.

15. Impact, Limitations, and Next Steps

Expected impact

Earlier detection

Surface pipeline, freshness, and data-quality issues earlier in the operational workflow.

Evidence-driven investigation

Show structured findings with traceable source references so users can validate what the system reports.

Exasol-powered analytics

Use SQL views, aggregations, and deterministic rules to evaluate operational data consistently.

More connected decisions

Bring pipeline health, data trust, requests, and cost context into one operating view.

Limitations to communicate clearly

- The dashboard can only display information available through configured connections and permissions.
- Cloud cost APIs can be delayed, throttled, or restricted by access policies.
- AI-generated SQL may be incorrect and must be validated before execution.
- The local validation figures in the pitch deck are a point-in-time demonstration, not a production service-level guarantee.

Next steps

16. Re-run validation on the current commit and preserve the evidence output.
17. Improve source freshness and degraded-state indicators.
18. Expand test coverage for permissions, timeouts, empty results, and malformed responses.
19. Harden generated-SQL validation and read-only access controls.
20. Complete contributor profiles, license, and reproducibility documentation.

16. Conclusion — Thank You

Detect the signal. Validate the evidence. Enable better decisions.

OpsNexus brings operational signals into a shared view so data teams can investigate pipeline activity, assess data trust, support requests, and understand platform cost with greater context.

Exasol Personal provides the analytical core; React and Flask connect the user experience to data and integrations; Azure supplies operational and cost context; and Ollama/Llama 3 offers an optional natural-language assistance path.

Thank you.

Project

https://github.com/vyshhnavi-ndd/OpsNexus/tree/main

Team Nexus • Exathon 2026 • Track 04 — Work Behind the Work